Subject: ACD Faculty: Bolleddu Devananda Rao Topic: Central concept of Finite Automata	**Class Notes**	Unit No: 1 Lecture No: Link to Session Planner (SP): Book Reference: Date Conducted: Page No: 1

Fundamentals

Alphabet:- An alphabet is a finite non-empty set of symbols conventionally we use the symbol 'Σ' for an alphabet.

1. $\Sigma = \{0, 1\}$ binary alphabet
2. $\Sigma = \{a, b, c\}$
3. $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
4. $\Sigma = \{a, b, -- z\}$

Strings:- A string (word) is a finite sequence of symbols choosen from some alphabets. For example: 01101 is a string from the binary alphabet $\Sigma = \{0, 1\}$ The string '111' is another string choosen from this alphabet.

Empty String:- The empty string is a string with "zero occurrences" of symbols. This string, denoted by 'ε', is a string that may be choosen from any alphabet. what so ever.

Length of the ~~sym~~ string:- Is often useful to classify strings by their length i.e., "the no. of positions for symbols in the string". For instance 01101 has length '5'

→ The standard notations for the length of the string w is $|w|$

~~Eg~~ example: ① $\Sigma = \{0, 1\}$

$w = 01101$

$|w| = 5$

② $|011| = 3$

$|010| = 3$

$|\varepsilon| = 0.$

The length of the epsilon is zero.

Powers of an alphabet :- If 'Σ' is an alphabet we can express the set of all strings of a certain length from that alphabet by using an exponential notation.

we define Σ^k to be the set of strings of length 'k', each of whose symbols is in Σ.

Example:- Note that $\Sigma^0 = \{\varepsilon\}$ no matter what the alphabet 'Σ' is. In otherwords 'ε' is the only string of length zero.

① $\Sigma = \{0, 1\}$

$2^1 = \quad \Sigma^1 = \{0, 1\}$

$2^2 = \quad \Sigma^2 = \{00, 01, 10, 11\}$

$2^3 = \quad \Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$

② $\Sigma = \{a, b, c\}$ is an alphabet

$\Sigma^1 = \{a, b, c\}$ set of strings whose length is 1.

$\Sigma^2 = \{aa, ab, ac, ba, bb, bc, ca, cb, cc\}$

The set of all strings over an alphabet 'Σ' is conventionally denoted by 'Σ^*' for instance

$$\{0,1\}^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, 001, \ldots\}$$

we can write it in another way.

$$\Sigma^* = \{\Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \ldots\}$$

$$\Sigma^+ = \Sigma^* - \{\varepsilon\}$$

LANGUAGE :- A set of strings all are choosen from Σ^* where Σ is a particular alphabet, is called a 'Language'

→ If Σ is an alphabet and $L \subseteq \Sigma^*$ then L is a language over 'Σ'

→ Notice that a language over 'Σ' need not include strings with all the symbols of 'Σ'.

Example :- ① The language of all strings consisting of n 0's followed by n 1's for some $n \geq 0$.

$$\Sigma = \{0,1\} \qquad L \subseteq \Sigma^*$$

$$L = \{\varepsilon, 01, 0011, 000111, 00001111, 0000011111, \ldots\}$$

when $n=0$ ~~at~~ Language is 'ε'

② The set of strings of 0's and 1's with equal no. of each $\Sigma = \{0, 1\}$

$L = \{\varepsilon, 01, 10, 1100, 1010, 0011, \text{---}\}$

③ The set of binary numbers whose value is a prime

$$L = \{\underset{2}{10}, \underset{3}{11}, \underset{5}{101}, \underset{7}{111}, \underset{11}{1011}, \text{----}\}$$

④ Σ^* is a language for any alphabet 'Σ'

ϕ, the empty language, is a language over any alphabet

$L = \{\ \}$

$L = \phi$

⑤ The language consisting of only the empty string is also a language over any alphabet.

Notice that $\underset{\text{empty set}}{\phi} \neq \underset{\text{set with empty string}}{\{\varepsilon\}}$

→ A language is a subset of the set of strings over an alphabet.

→ A language can be recognized by a machine, such a device is called a recognition device.

→ A language can have a representation interms of a generating device as a grammar as well as interms of a recognition device which is called an acceptor.

→ The simplest device & recognition device is the finite state automaton.

→ Consider a man watching a TV in his room. The TV is in 'ON' state. when it is switched off, the TV goes to "off" state. when it is switched on, it again goes to 'ON' state. This can be represented by the following figure.

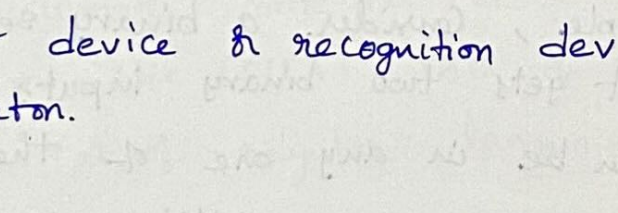


fig: An example of FSA.

→ The above diagram is called a state diagram.

→ Consider another example, a set of processes currently on a single processor, Some Scheduling algorithm allots them time on the processor. The states in which a system can be are 'wait', 'Run', 'Start', 'end'. The Connections between them can be brought out by the following figure.

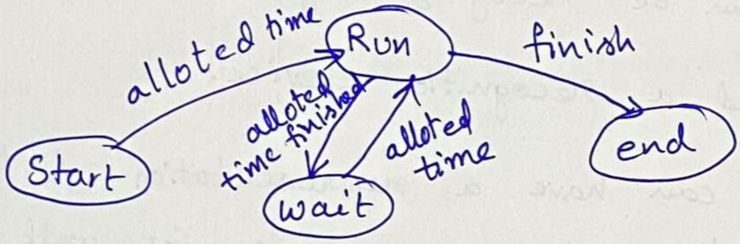


Fig: Another example of FA

→ Another example, Consider a binary serial adder. At any time it gets two binary inputs x_1 and x_2: the adder can be in any one of the states 'carry' & 'no carry'. The four possibilities for the inputs $x_1 x_2$ are 00, 01, 10, and 11. Initially, the adder is in the "no carry" state. The working of the serial adder can be represented by the following figure

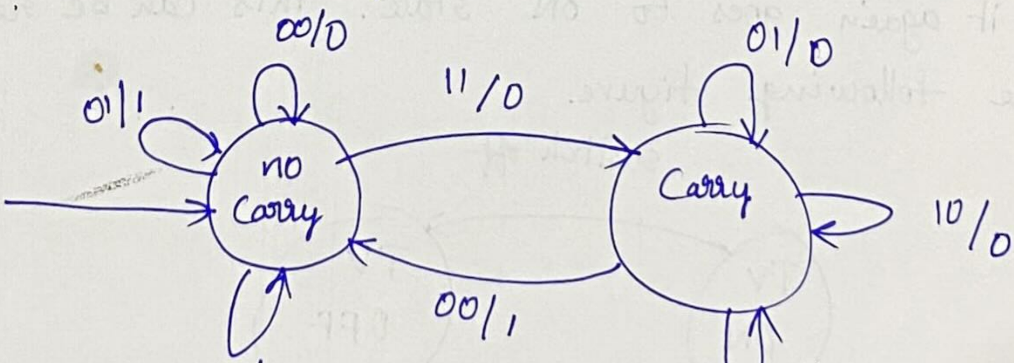


Figure: Binary Serial adder

* Finite automata are abstract machines used to recognize patterns in input sequences.

* They Consists of states, transitions, and input symbols, processing each symbol step-by-step.

* If the machine ends in an accepting state after processing the input, it is accepted otherwise it is rejected.

* Finite Automata come in deterministic (DFA) and non-deterministic (NFA), both of which can recognize the same set of regular languages.

* They are widely used in text processing, compilers, and network protocols.

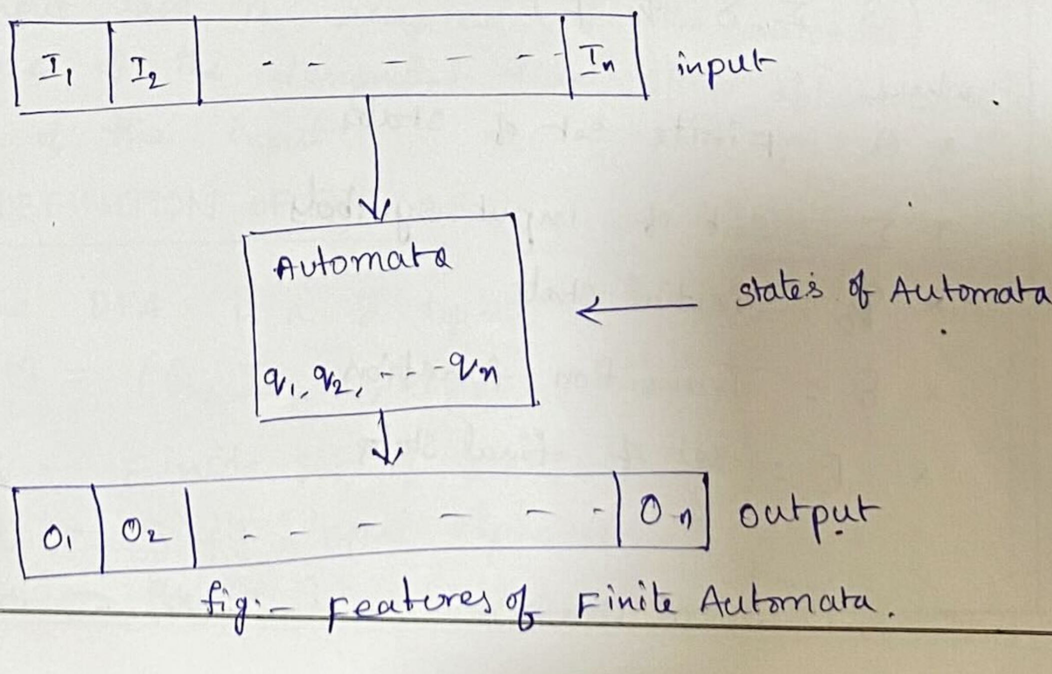


fig:- features of Finite Automata.

Features of Finite Automata

* Input :- set of symbols (or) characters provided to the machine

* Output :- Accept or reject based on the input pattern

* states of Automata :- The conditions or configurations of the machine

* state Relation :- The Transitions between the states.

* output Relation :- Based on the final state, the output decision is made.

Formal Definition of Finite Automata

A Finite Automaton can be defined as a 5 tuple.
$(Q, \Sigma, \delta, q_0, F)$

where

* Q : Finite set of states
* Σ : set of input symbols
* q_0 : initial state
* δ = Transition function
* F = set of final states

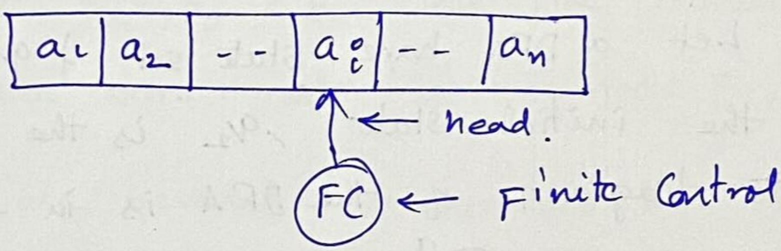
Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Deterministic Finite state Automata (DFSA)

Class Notes

Unit No: 1
Lecture No:
Link to Session Planner (SP):
Book Reference:
Date Conducted:
Page No: 5

→ The Finite state automata (FSA) as accepting languages i.e., stet of strings.

→ we can look at the FSA with an input tape, a tape head & a finite control which denotes the state.



→ The input string is placed on the input tape, each cell contains one symbol, the tape head initially points to the leftmost symbol. At any stage depending on the stage and the symbol read the automata changes its state. and moves its tape head 1 cell to the right. the string on the input tape is accepted if the automata goes to one of the designated final state after reading the whole of the input.

FORMAL DEFINITION OF THE DFA is as follows

Def :- The DFA is a 5 tuple

$$M = (Q, \Sigma, \delta, q_0, F)$$

where Q — Finite set of states

Σ — Finite input alphabet

q_0 — q_0 in Q is initial state / start state.

F — $F \subseteq Q$ is set of final states.
δ — Transition function / mapping.
$\delta : Q \times \Sigma \rightarrow Q$.

$\delta(q, a) = p$.

If the automata is in state 'q' & reading a symbol 'a' it goes to state 'p' in the next instant, moving pointer one cell to the right.

<u>Example</u>:- Let a DFA have state set $\{q_0, q_1, q_2, D\}$, q_0 is the initial state, q_2 is the only final state. the state diagram of the DFA is in fig.

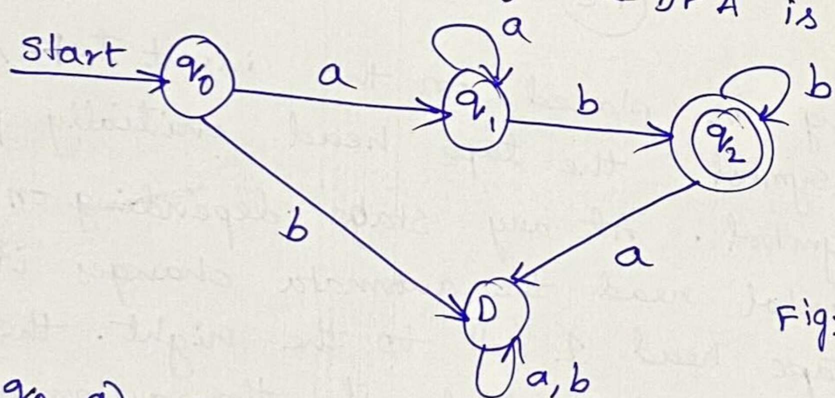


Fig:- DFA accepting $\{a^n b^m / m, n \geq 1\}$

$\delta(q_0, a) = q_1$
$\delta(q_0, b) = D$
$\delta(q_1, a) = q_1$
$\delta(q_1, b) = q_2$
$\delta(q_2, a) = D$
$\delta(q_2, b) = q_2$

aaabb
↑
q_0

aaabb
↑
q_1

aaabb
↑
q_1

aaabb
↑
q_1

aaabb
↑
q_2

aaabb
↑
q_2

Transition Diagram for a DFA $M = (Q, \Sigma, \delta, q_0, F)$ is a graph defined as follows.

(a) For each state in 'Q' there is a 'node' in the diagram

(b) → For each state q in Q & each input symbol 'a' in Σ, Let $\delta(q, a) = p$. then the transition diagram has an arc from node 'q' to node 'p', labelled 'a'.

→ If there are several i/p symbols that cause ~~to~~ transition from q to p, then the transition diagram can have one ~~labelled~~ arc labelled by the list of these symbols.

(c) there is an arrow into the start state q_0, labelled 'start' this arrow does not originate at any node

(d) Nodes corresponding to accepting state (those in F) are marked by "double circle", states not in F have a single circle

Ex:- Fig shows the Transition diagram for the DFA, that accepts all and only the strings of 0's & 1's that have the sequence '01' some where in the string.

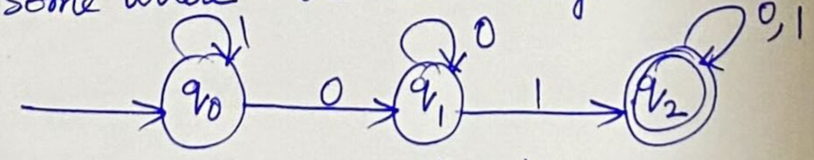


Fig:- DFA accepting all strings containing 01 as substring

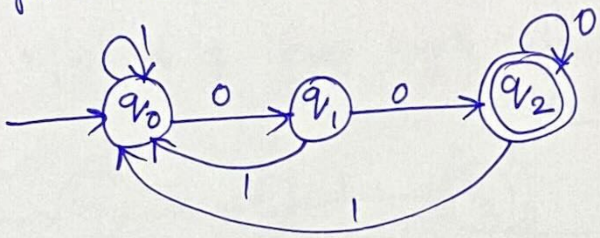
Transition Table :- A Transition Table is a conventional Tabular representation of a function like 'δ' that takes two arguments & return a value.

→ The rows of the table corresponds to the states and the column corresponds to the input. the entry for the row corresponding to state 'q' and the column corresponding to input 'a' is the state $\delta(q, a)$

Transition Table

δ	0	1
→ q_0	q_1	q_0
q_1	q_1	q_2
* q_2	q_2	q_2

① Design a DFA that accepts all strings ending with '00'.



Transition Table

δ	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_2	q_0
$*q_2$	q_2	q_0.

DFA. $M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{0, 1\}$

$\delta = Q \times \Sigma \rightarrow Q$.

q_0 – initial state

$F = \{q_2\}$

$\delta(q_0, 0) = q_1$

$\delta(q_0, 1) = q_0$

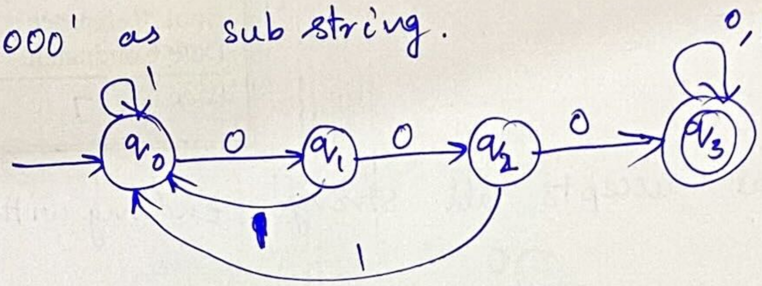
$\delta(q_1, 0) = q_2$

$\delta(q_1, 1) = q_0$

$\delta(q_2, 0) = q_2$

$\delta(q_2, 1) = q_0$

(2) Design a DFA that accepts all strings containing '000' as sub string.



δ	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_2	q_0
q_2	q_3	q_0
$*q_3$	q_3	q_3

$\delta(q_0, 0) = q_1$

$\delta(q_0, 1) = q_0$

$\delta(q_1, 0) = q_2$

$\delta(q_1, 1) = q_0$

$\delta(q_2, 0) = q_3$

$\delta(q_2, 1) = q_0$

$\delta(q_3, 0) = q_3$

$\delta(q_3, 1) = q_3$

DFA $M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{0, 1\}$

q_0 — initial state

$F = \{q_3\}$

$\delta : Q \times \Sigma \rightarrow Q$.

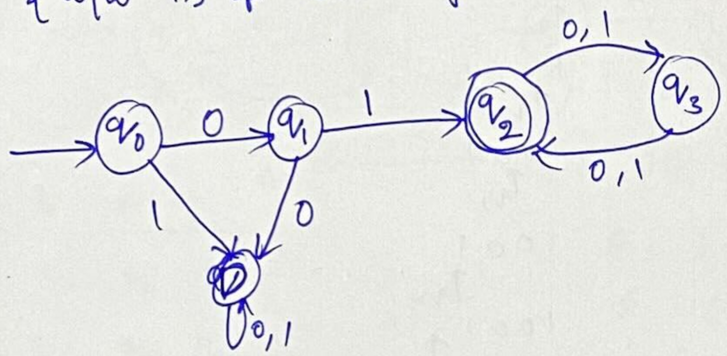
Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Design of DFA

Class Notes

Unit No: 1
Lecture No:
Link to Session Planner (SP):
Book Reference:
Date Conducted:
Page No: 8

③ Let us design a DFA to accept the language

$T(M) = \{ w | w$ is of even length and begins with '01' $\}$



Transition Table

δ	0	1
$\rightarrow q_0$	q_1	D
q_1	D	q_2
$* q_2$	q_3	q_3
q_3	q_2	q_2
D	D	D.

DFA $M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{ q_0, q_1, q_2, q_3, D \}$

$\Sigma = \{0, 1\}$

$\delta : Q \times \Sigma \rightarrow Q$

q_0 – initial state

$F = \{ q_2 \}$

$\delta(q_0, 0) = q_1$

$\delta(q_0, 1) = D$

$\delta(q_1, 0) = D$

$\delta(q_1, 1) = q_2$

$\delta(q_2, 0) = q_3$

$\delta(q_2, 1) = q_3$

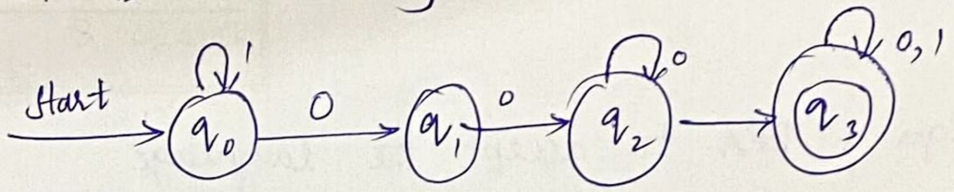
$\delta(q_3, 0) = q_2$

$\delta(q_3, q_2) = q_2$

$\delta(D, 0) = D$

$\delta(D, 1) = D.$

4) Design a DFA that accepts all strings containing 001 as a substring over $\Sigma = \{0,1\}$



δ	0	1
q_0	q_1	q_0
q_1	q_2	q_0
q_2	q_2	q_3
q_3	q_3	q_3

1001 ↑q_0

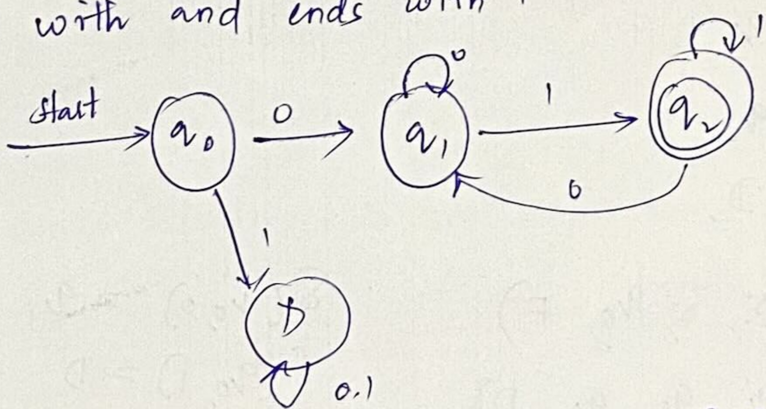
1001 ↑q_1

1001 ↑q_1

1001 ↑q_2

1001 ↑q_3

5) Construct transition graph for a finite automata which accepts the language 'L' over $\Sigma = \{0,1\}$ in which every string starts with 0 and ends with 1



$Q = \{q_0, q_1, q_2, D\}$

$\Sigma = \{0,1\}$

$q_0 = \{q_0\}$

$f = q_2$

DFA $M = (Q, \Sigma, \delta, q_0, f)$

δ	0	1
q_0	q_1	D
q_1	q_1	q_2
q_2	q_1	q_2
D	D	D

00101 ↑q_0

00101 ↑q_1

00101 ↑q_1

00101 ↑q_2

00101 ↑q_2

00101 ↑q_2

q_2

Subject: ACD

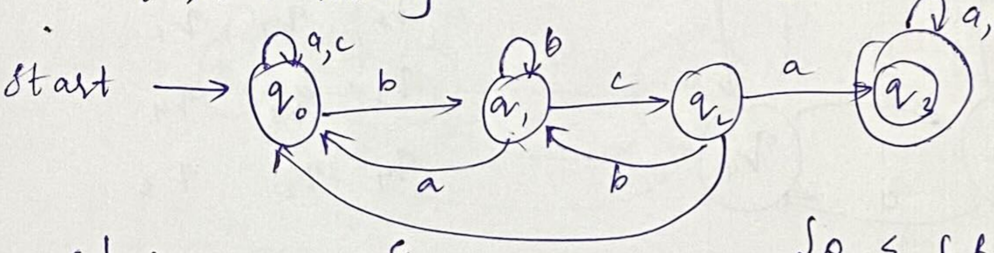
Faculty: Bolleddu Devananda Rao

Topic: Design of DFA

Class Notes

Unit No: 1
Lecture No:
Link to Session Planner (SP):
Book Reference:
Date Conducted:

(6) Construct DFA for the set of strings over $\Sigma = \{a, b, c\}$ having bca as substring



δ	b	c	a
q_0	q_1	q_0	q_0
q_1	q_1	q_2	q_0
q_2	q_1	q_0	q_3
q_3	q_3	q_3	q_3

$\{Q, \Sigma, \delta, f, q_0\}$

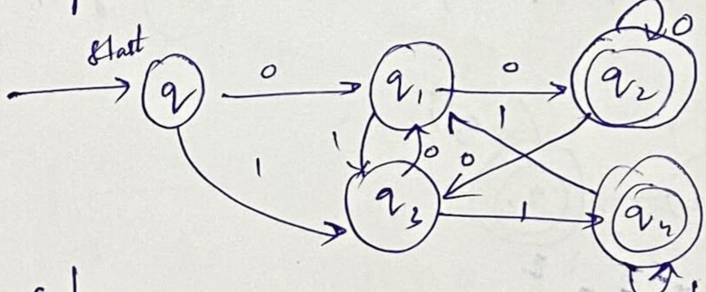
$Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = Q \times \Sigma \to Q$

$f = q_3$

q_0 = Initial

(7) Design a DFA to accept the language $T(M) = \{x \in \{0,1\}^* \mid$ string x ends with either '00' or '11'



δ	0	1
q_0	q_1	q_3
q_1	q_2	q_3
q_2	q_2	q_3
q_3	q_1	q_4
q_4	q_1	q_4

$M = \{Q, \Sigma, \delta, f, q_0\}$

$Q = \{q_0, q_1, q_2, q_3\}$

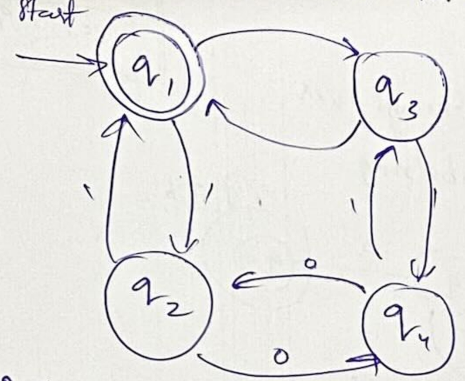
$\Sigma = \{0, 1\}$

$\delta = Q \times \Sigma \to Q$

$f = q_2, q_4$

$q_0 = q_0.$

(7) Construct DFA for set of strings over $\Sigma = \{0,1\}$ having even no. of 0's and even no. of 1's



δ	0	1
q_1	q_3	q_2
q_2	q_4	q_1
q_3	q_1	q_4
q_4	q_2	q_3

$M = \{Q, \Sigma, \delta, f, q_1\}$

$Q = \{q_1, q_2, q_3, q_4\}$

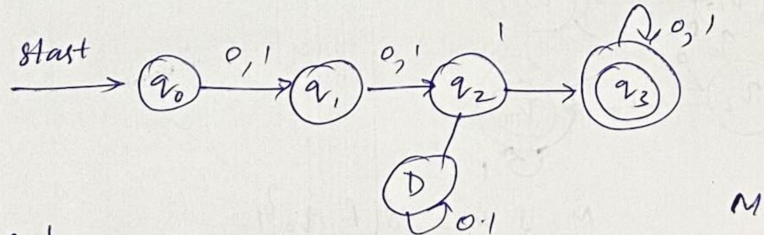
$\Sigma = \{0,1\}$

$F = q$

$q_0 = q$ {Initial state)

$\delta = Q \times \Sigma \rightarrow Q$.

(8) Construct DFA for a language "L" which accepts all strings in which the third symbol from left end is always 4 over $\Sigma = \{0,1\}$



δ	0	1
q_0	q_1	q_1
q_1	q_2	q_2
q_2	D	q_3
q_3	q_3	q_3

$M = \{Q, \Sigma, \delta, F, q_0\}$

$Q = \{q_0, q_1, q_2, q_3, D\}$

$\Sigma = \{0,1\}$

$\delta = Q \times \Sigma \rightarrow Q$

$f = q_3$

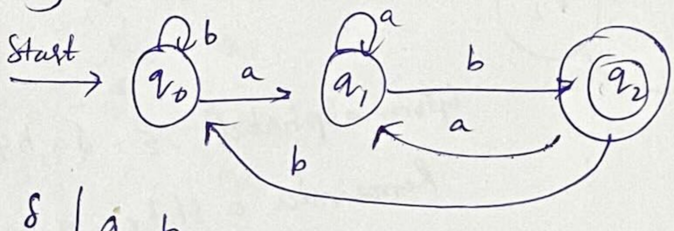
q_0 · Initial State

Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Design of DFA.

Class Notes

Unit No: 1
Lecture No:
Link to Session Planner (SP):
Book Reference:
Date Conducted:
Page No: 10

(9) Construct DFA accepting strings over $\Sigma = \{a,b\}$ ending in a,b



$M = \{Q, \Sigma, \delta, F, q_0\}$

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{a,b\}$

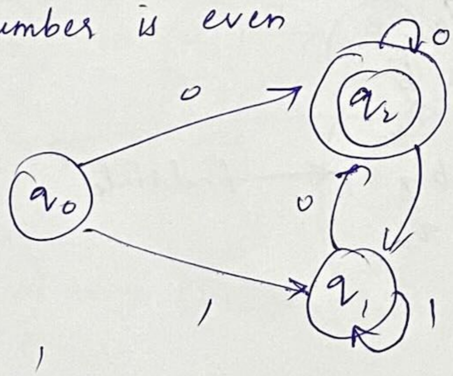
$F = q_2$

q_0 = Initial State.

$\delta = Q \times \Sigma \rightarrow Q$

δ	a	b
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_0

(10) Design a finite automata which checks whether the binary number is even



$\Sigma = \{0,1\}$

$M = \{Q, \Sigma, \delta, F, q_0\}$

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{0,1\}$

$F = q_2$

q_0 = Initial State

$\delta = Q \times \Sigma \rightarrow Q$.

δ	0	1
q_0	q_2	q_1
q_1	q_2	q_1
q_2	q_2	q_1

1100
$\uparrow q_0$

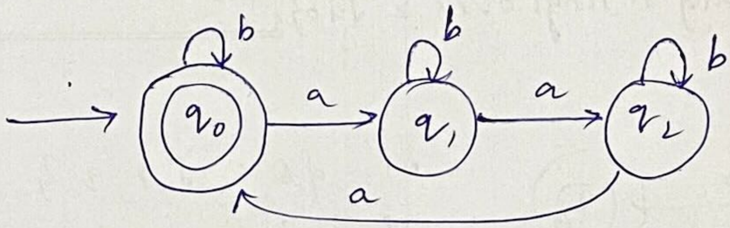
1100
$\uparrow q_1$

1100
$\uparrow q_1$

1100
$\uparrow q_2$

1100 $\uparrow q_2$ → final state.

(11) Design a DFA to accept language where all the strings in 'L' such that total no. of a's in them are divisible by '3'



Divisible by 3

remainder '0'
'1'
'2'

Given alphabet $\Sigma = \{a, b\}$

Remainder '0' state - q_0
'1' state - q_1
'2' state - q_2

$M = \{Q, \Sigma, \delta, f, q_0\}$

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{a, b\}$

$f = q_0$

q_0 = Initial state

$\delta = Q \times \Sigma \rightarrow Q$

δ	a	b
q_0	q_1	q_0
q_1	q_2	q_1
q_2	q_0	q_2

a a b a b
$\uparrow q_0$

a a b a b
$\uparrow q_1$

a a b a b
$\uparrow q_2$

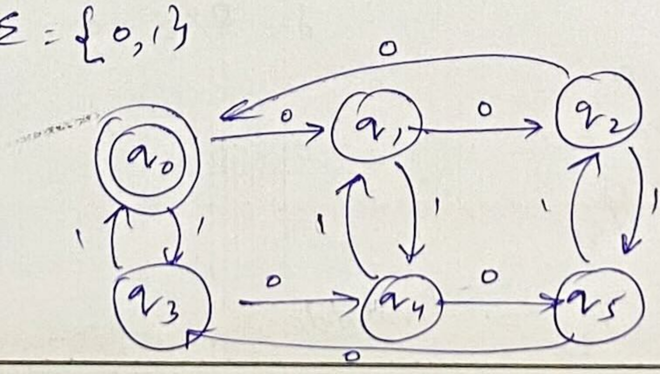
a a b a b
$\uparrow q_2$

a a b a b
$\uparrow q_0$

a a b a b $\uparrow$ q_0 ← final state

(12) Design a DFA for the set of all strings such that the number of 1's is even and 0's are multiples of 3

$\Sigma = \{0, 1\}$



$M = \{Q, \Sigma, \delta, f, q_0\}$

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$

$\Sigma = \{0, 1\}$

$F = q_0$ - Initial

$\delta = Q \times \Sigma \rightarrow Q$.

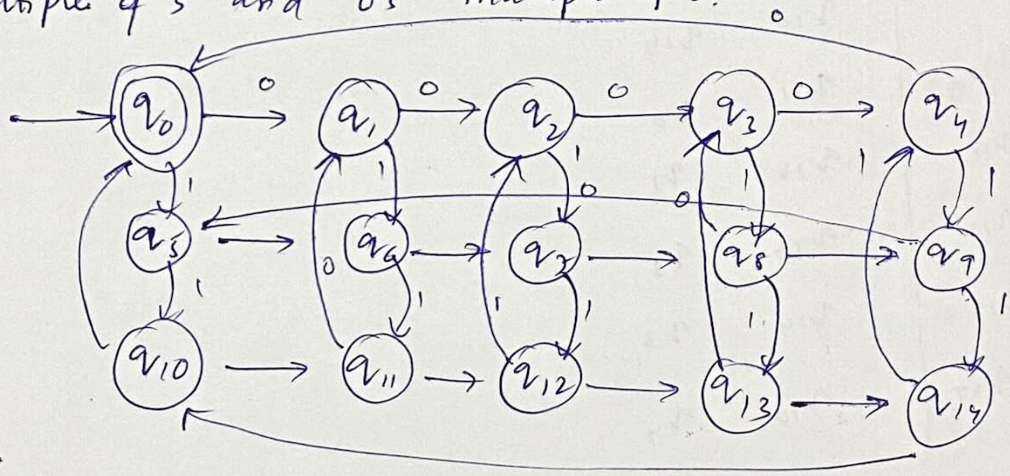
Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Design of DFA

Class Notes

Unit No: 1
Lecture No:
Link to Session
Planner (SP):
Book Reference:
Date Conducted:
Page No: 11

δ	0	1
q_0	q_1	q_3
q_1	q_2	q_4
q_2	q_0	q_5
q_3	q_4	q_0
q_4	q_5	q_1
q_5	q_3	q_2

(13). Construct a DFA for all strings that accepts 1's multiple of 3 and 0's multiples of 5.



10101000 ↑ q_0

10101000 ↑ q_5

10101000 ↑ q_6

10101000 ↑ q_{11}

10101000 ↑ q_{12}

10101000 ↑ q_2

10101000 ↑ q_3

10101000 ↑ q_4

10101000 ↑ q_0 → final state

$M = \{Q, \Sigma, \delta, f, q_0\}$

$Q = \{q_1, q_0, \ldots q_{14}\}$

$\Sigma = \{0, 1\}$

$f = q_0$

$I = q_0$

$\delta = Q \times \Sigma = q_0.$

δ	0	1
q_0	q_1	q_5
q_1	q_2	q_6
q_2	q_3	q_7
q_3	q_4	q_8
q_4	q_0	q_9
q_5	q_6	q_{10}
q_6	q_7	q_{11}
q_7	q_8	q_{12}
q_8	q_9	q_{13}
q_9	q_{10}	q_{14}
q_{10}	q_{11}	q_0
q_{11}	q_{12}	q_1
q_{12}	q_{13}	q_2
q_{13}	q_{14}	q_3
q_{14}	q_{10}	q_4

(14) Design a DFA for the language

$T(M) = \{ x \mid \text{the sum of digits in } x \text{ is } 2 \bmod 3 \}$

$\Sigma = \{0, 1, 2\}$

$M = \{Q, \Sigma, \delta, f, q_0\}$

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{0, 1, 2\}$

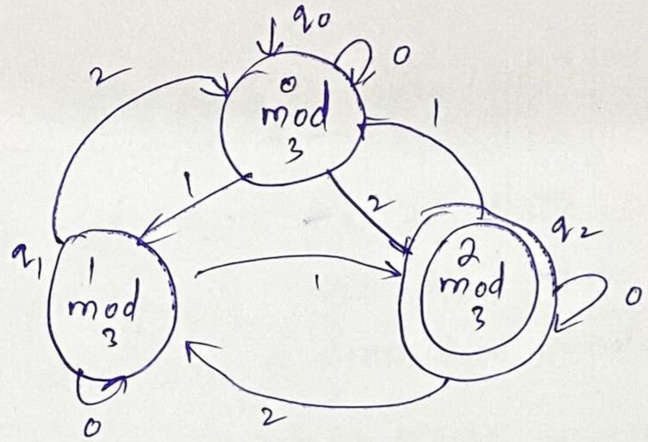
$f = q_2$

$I = q_0$

$\delta = Q \times \Sigma \rightarrow Q$

Class Notes

Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: DFA



$M = \{Q, \Sigma, \delta, f, q_0\}$

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{0, 1, 2\}$

$f = q_2$

$I = q_0$

$\delta = Q \times \Sigma \to Q.$

1 0 1 2 0 2 1 1
↑ q_0

1 0 1 2 0 2 1 1
↑ q_1

1 0 1 2 0 2 1 1
↑ q_1

1 0 1 2 0 2 1 1
↑ q_2

1 0 1 2 0 2 1 1
↑ q_1

1 0 1 2 0 2 1 1
↑ q_1

1 0 1 2 0 2 1 1
↑ q_0

1 0 1 2 0 2 1 1
↑ q_1

1 0 1 2 0 2 1 1 ↑ q_2 → final

Subject: ACD

Faculty: Bolleddu Devananda Rao

Topic: Non-deterministic finite Automata

Class Notes

Unit No: 1
Lecture No:
Link to Session
Planner (SP):
Book Reference:
Date Conducted:
Page No: 13

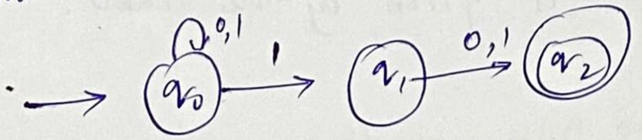
Non-Deterministic finite Automata (NFA)

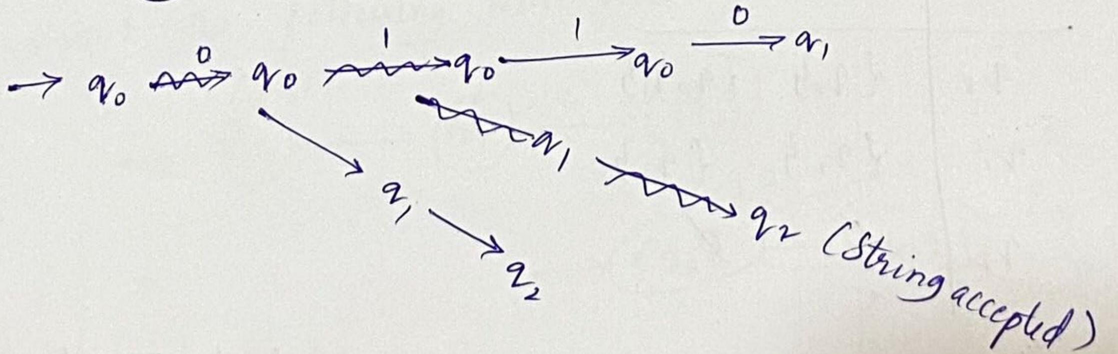
Like the 'DFA', 'NFA' has a finite set of states a finite set of input symbols, one start state and a set of accepting states, it also has a transition function 'delta'

→ The difference of DFA & NFA is the type of delta for the NFA, 'δ' is a function that takes a state & input symbol as arguments but returns a set of zero, 1 (or) more states (rather than returning exactly 1 state as DFA)

Ex:

Figure shows the NFA which accepts exactly those strings that have the symbol '1' in the second last position.





Formal defination of NFA

NFA is a 5 tuple, $M = \{Q, \Sigma, \delta, q_0, F\}$ where 'Q' is finite set of States, 'Σ' is finite set of input alphabet, 'q_0' in Q is final state, 'F' is Subset of 'Q' in the set of final states.

δ = Transition function is a function that takes a state in 'Q' & the input symbol is 'Σ' as arguments and returns a subset of 'Q'.

Notice that the only difference between NFA & DFA is in the type of value that 'δ' returns, a set of states in the case of NFA & a single state in case of DFA.

$$\delta : Q \times \Sigma \rightarrow Q \quad - \text{ DFA.}$$

<u>NFA:</u>

$$\delta : Q \times \Sigma \rightarrow 2^{Q} \quad (\text{power set of } 'Q')$$

Ex:- The NFA of the above figure can be specified formally as $(\{q_0, q_1, q_2\}\ \{0,1\}\ \delta .\ \{q_0\}\ \{q_2\})$ where the transition function 'δ' is given by the table.

δ	'0'	'1'
q_0	$\{q_0\}$	$\{q_0 q_1\}$
q_1	$\{q_2\}$	$\{q_2\}$
q_2	$\emptyset$	$\emptyset$

Notice that, transition tables can be used to specify the transition function for an NFA as well as an DFA.

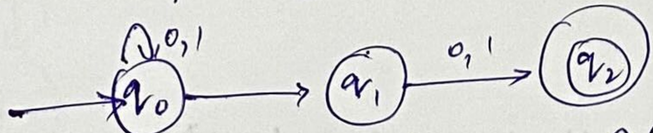
→ The only difference is that each entry in the table for a NFA is a set, even the set is a single term (as one member) & also notice that when there is no transition at all from a given state on a given input symbol the proper entry is ϕ. (empty set)

Design of NFA

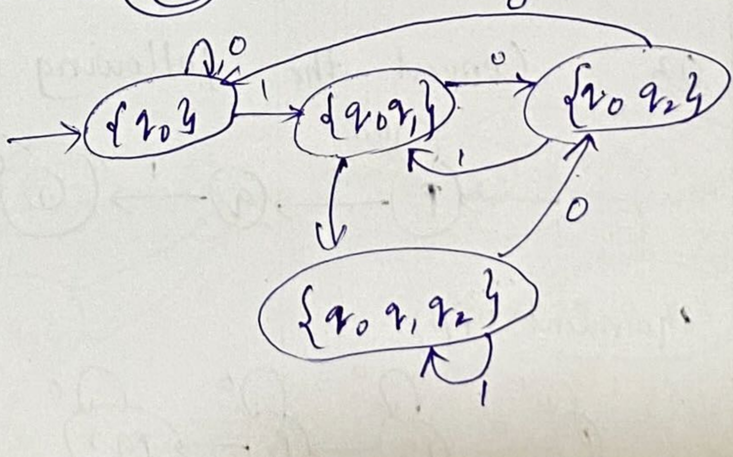
Equivalence of DFA & NFA:

Every language that can be described by NFA can also be described by DFA. DFA has as many states as NFA although it often has more transitions. In worst case, the smallest DFA can has 2^n states. while the smallest NFA for the smallest language has only 'n' states.

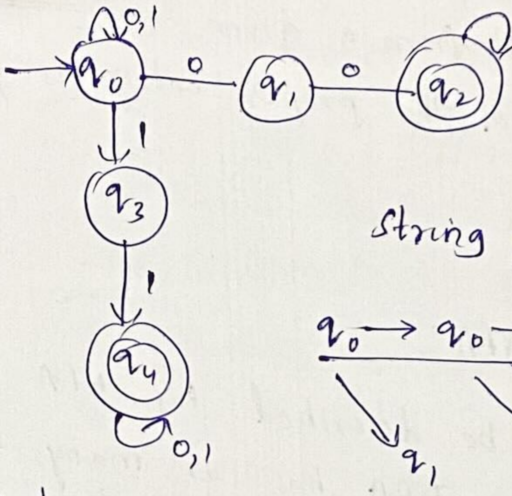
Q1 Convert the following NFA into DFA.



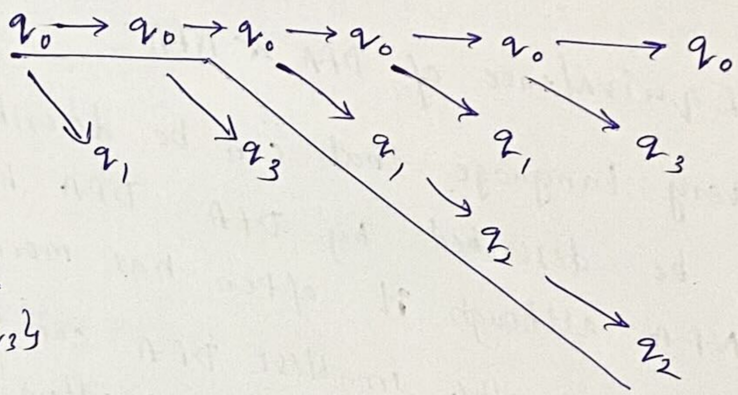
δ	0	1
$\rightarrow\{q_0\}$	$\{q_0\}$	$\{q_0 q_1\}$
$\{q_0 q_1\}$	$\{q_0 q_2\}$	$\{q_0 q_1, q_2\}$
$*\{q_0 q_2\}$	$\{q_0\}$	$\{q_0 q_1\}$
$*\{q_0 q_1, q_2\}$	$\{q_0 q_2\}$	$\{q_0 q_1, q_2\}$



Q2 The state diagram of NFA which have atleast one pair of 0 (0,0) or one pair of one (1,1) is in figure.

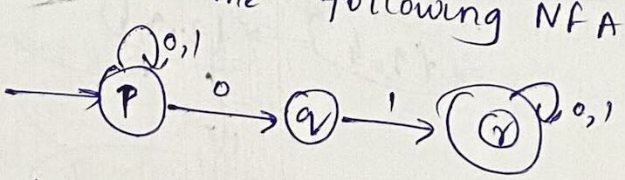


String processing : 01001

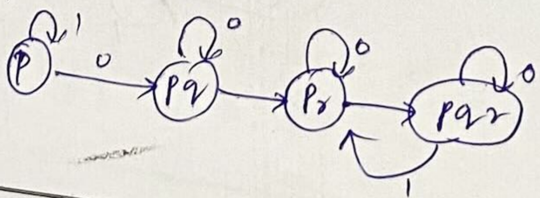


δ	0	1
$\{q_0\}$	$\{q_0 q_1\}$	$\{q_0 q_3\}$
$\{q_1\}$	$\{q_2\}$	$\emptyset$
$\{q_2\}$	$\{q_2\}$	$\{q_2\}$
$\{q_3\}$	$\emptyset$	$\{q_4\}$
$\{q_4\}$	$\{q_4\}$	$\{q_4\}$

Q3. Convert the following NFA to DFA.



equivalent DFA :-

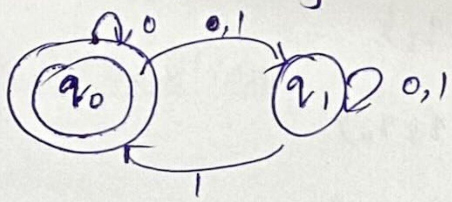


	0	1
$\{p\}$	$\{pq\}$	$\{p\}$
$\{pq\}$	$\{pq\}$	$\{pr\}$
$*\{pr\}$	$\{pqr\}$	$\{pr\}$
$*\{pqr\}$	$\{pqr\}$	$\{pr\}$

Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Design of NFA

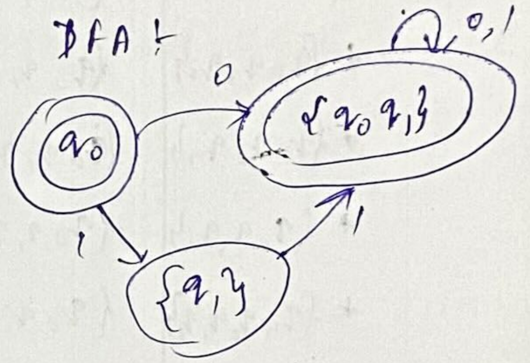
Class Notes

Q4 Convert the following NFA to DFA

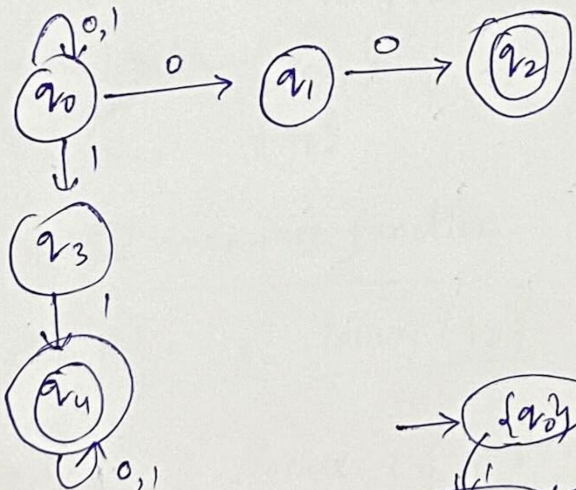


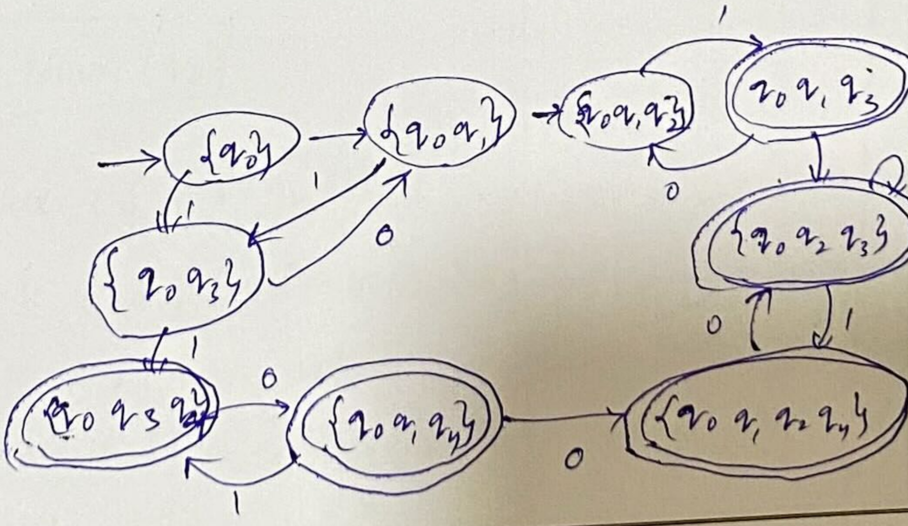
	0	1
* $\{q_0\}$	$\{q_0 q_1\}$	$\{q_1\}$
* $\{q_0 q_1\}$	$\{q_0 q_1\}$	$\{q_1, q_0\}$
$\{q_1\}$	$\{q_1\}$	$\{q_0 q_1\}$

DFA :-



Q5. Convert NFA to DFA





	0	1
$\rightarrow \{q_0\}$	$\{q_0 q_1\}$	$\{q_0 q_3\}$
$\{q_0 q_1\}$	$\{q_0 q_1 q_2\}$	$\{q_0 q_3\}$
$\{q_0 q_3\}$	$\{q_0 q_1\}$	$\{q_0 q_3 q_4\}$
$*\{q_0 q_1 q_2\}$	$\{q_0 q_1 q_2\}$	$\{q_0 q_3 q_1\}$
$*\{q_0 q_3 q_2\}$	$\{q_0 q_1 q_2\}$	$\{q_2 q_0 q_3\}$
$*\{q_0 q_1 q_4\}$	$\{q_0 q_1 q_2 q_4\}$	$\{q_0 q_3 q_4\}$
$*\{q_0 q_3 q_4\}$	$\{q_0 q_1 q_4\}$	$\{q_0 q_3 q_4\}$
$*\{q_4 q_0 q_1 q_2\}$	$\{q_0 q_1 q_2 q_4\}$	$\{q_0 q_3 q_2 q_4\}$
$*\{q_0 q_3 q_2 q_4\}$	$\{q_0 q_1 q_2 q_4\}$	$\{q_0 q_3 q_4 q_2\}$

NFA with ϵ transitions

Formal Defination.

NFA with ϵ transitions is a 5 tuple $M = (Q, \Sigma, \delta, q_0, F)$

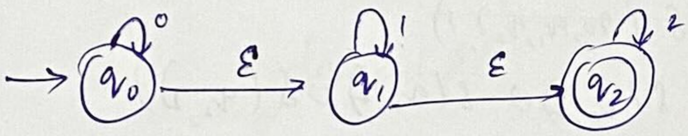
where Q is finite set of states

Σ is finite set of Input alphabet.

$\delta : Q \times \Sigma \cup \{\epsilon\} \rightarrow 2^Q$

q_0 is the initial state

F is set of final states



ϵ - closure $(q_0) = \{q_0 q_1, q_2\}$

ϵ - closure $(q_1) = \{q_1, q_2\}$

ϵ - closure $(q_2) = \{q_2\}$

$\hat{\delta}$ - extended transition function.

$$\boxed{\hat{\delta}(q_0, \epsilon) = \epsilon\text{-closure}(q_0)}$$

$\hat{\delta}(q_0, 0) = \epsilon$-closure $(\delta(\hat{\delta}(q_0, \epsilon)), 0)$

$= \epsilon$-closure $(\delta(\epsilon$-closure $(q_0), 0)$

$= \epsilon$-closure $(\delta(q_0, q_1, q_2), 0)$

$= \epsilon$-closure $(\delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0))$.

$= \epsilon$-closure $(q_0 \cup \phi \cup \phi)$

$= \varepsilon\text{-closure}(q_0)$

$= \{q_0, q_1, q_2\}$

$\hat{\delta}(q_0, 1) = \varepsilon\text{-closure}(\delta(\hat{\delta}(q_0, \epsilon)), 1)$

$= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_0), 1))$

$= \varepsilon\text{-closure}(\delta(q_0 q_1 q_2), 1)$

$= \varepsilon\text{-closure}(\delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1))$

$= \varepsilon\text{-closure}(\phi \cup q_1 \cup \phi)$

$= \varepsilon\text{-closure}(q_1)$

$= \{q_1, q_2\}$

$\hat{\delta}(q_0, 2) = \varepsilon\text{-closure}(\delta(q_0 q_1 q_2), 2)$

$= \cancel{\varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_0), 1)}$

$= \cancel{\varepsilon\text{-closure}(\delta(q_0, q_1, q_2), 1)}$

$= \varepsilon\text{-closure}(\delta(q_0, 2) \cup \delta(q_1, 2) \cup \delta(q_2, 2))$

$= \varepsilon\text{-closure}(\phi \cup q_2 \cup \phi)$

$= \varepsilon\text{-closure}(q_2)$

$= \{q_2\}$

$\hat{\delta}(q_1, 0) = \varepsilon\text{-closure}(\delta(\hat{\delta}(q_0, \epsilon)), 0)$

$= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_1), 0))$

$= \varepsilon\text{-closure}(\delta(q_1, q_2), 0)$

$= \varepsilon\text{-closure}(\delta(q_1, 0) \cup \delta(q_2, 0))$

$= \varepsilon\text{-closure}(\phi \cup \phi)$

$= \phi$

$\delta(q_1, 1) = \varepsilon\text{-closure}(\delta(q_1, q_2), 1)$

$= \varepsilon\text{-closure}(\delta(q_1, 1) \cup \delta(q_2, 1))$

$= \varepsilon\text{-closure}\ (q_1 \cup \phi)$

$= \{q_1, q_2\}$

$\delta(q_1, 2) = \varepsilon\text{-closure}\ (\delta(q_1 q_2), 2)$

$= \varepsilon\text{-closure}\ (\delta(q_1, 2) \cup \delta(q_2, 2))$

$= \varepsilon\text{-closure}\ (\phi \cup q_2)$

$= \{q_2\}$

$\delta(q_2, 0) = \varepsilon\text{-closure}\ (\delta(\hat{\delta}(q_0, \varepsilon)), 0)$

$= \varepsilon\text{-closure}\ (\delta\ (\varepsilon\text{-closure}(q_2), 0))$

$= \varepsilon\text{-closure}\ (\delta\ (q_2), 0)$

$= \varepsilon\text{-closure}\ (\delta(q_2, 0))$

$= \varepsilon\text{-closure}\ (\phi)$

$= \phi$

$\delta(q_2, 1) = \varepsilon\text{-closure}\ (\delta(q_2), 1)$

$= \varepsilon\text{-closure}\ (\delta(q_2, 1))$

$= \phi$

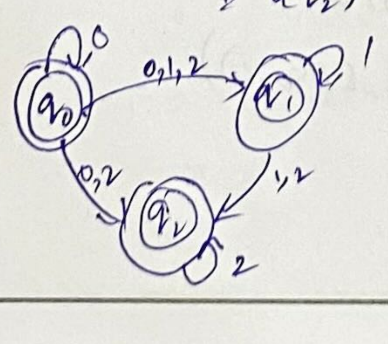
$\delta(q_2, 2) = \varepsilon\text{-closure}\ (\delta(\hat{\delta}(q_0, \varepsilon)), 2)$

$= \varepsilon\text{-closure}\ (\delta(\varepsilon\text{-closure}(q_2), 2)$

$= \varepsilon\text{-closure}\ (\delta(q_2), 2)$

$= \varepsilon\text{-closure}\ (\delta(q_2, 2))$

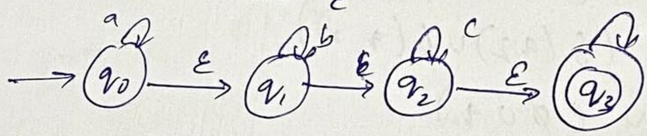
$= \{q_2\}$



δ	0	1	2
q_0	$\{q_0 q_1 q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
q_1	ϕ	$\{q_1, q_2\}$	$\{q_2\}$
q_2	ϕ	ϕ	$\{q_2\}$

Here q_0, q_1, q_2 are final states because ε-closure (q_0), ε-closure (q_1) & ε-closure (q_2) contains final state q_2.

Q. Convert the following NFA with ε into NFA without ε into NFA without ε



ε-closure $(q_0) = \{q_0, q_1\}$

ε-closure $(q_1) = \{q_1\}$

ε-closure $(q_2) = \{q_2\ q_3\}$

ε-closure $(q_3) = \{q_3\}$

$\hat{\delta}(q_0, a) = \varepsilon\text{-closure}(\delta(\hat{\delta}(q_0, \varepsilon)), a)$

$= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_0), a))$

$= \varepsilon\text{-closure}(\delta(q_0 q_1), a)$

$= \varepsilon\text{-closure}(\delta(q_0, a) \cup \delta(q_1, a))$

$= \varepsilon\text{-closure}(q_0 \cup \emptyset)$

$= \{q_0, q_1\}$

$\hat{\delta}(q_0, b) = \varepsilon\text{-closure}(\delta(q_0, q_1), b)$

$= \varepsilon\text{-closure}(\delta(q_0, b) \cup \delta(q_1, b))$

$= \varepsilon\text{-closure}((\emptyset \cup q_1)\ q_2)$

$\hat{\delta}(q_0, c) = \varepsilon\text{-closure}(\delta(q_0, q_1), c)$

$= \varepsilon\text{-closure}(\delta(q_0, c) \cup \delta(q_1, c))$

$= \varepsilon\text{-closure}(\emptyset \cup \emptyset)$

$= \emptyset$

Unit No: 1
Lecture No:
Link to Session
Planner (SP):
Book Reference:
Date Conducted:
Page No: 18

Class Notes

Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic:

$\hat{\delta}(q_0, d) = \varepsilon\text{-closure}(\delta(q_0 q_1), d)$
$= \varepsilon\text{-closure}(\delta(q_0, d) \cup \delta(q_1, d))$
$= \phi$

$\hat{\delta}(q_1, a) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_1), a))$
$= \varepsilon\text{-closure}(\delta(q_1), a)$
$= \varepsilon\text{-closure}(\delta(q_1, a))$
$= \phi$

$\hat{\delta}(q_1, b) = \varepsilon\text{-closure}(\delta(q_1), b)$
$= \varepsilon\text{-closure}(\delta(q_1, b))$
$= \varepsilon\text{-closure}(q_1, q_2)$

$\hat{\delta}(q_1, c) = \varepsilon\text{-closure}(\delta(q_1), c)$
$= \varepsilon\text{-closure}(\delta(q_1, c))$
$= \phi$

$\hat{\delta}(q_1, d) = \varepsilon\text{-closure}(\delta(q_1, d))$
$= \phi$

$\hat{\delta}(q_2, a) = \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(q_2), a)$
$= \varepsilon\text{-closure}(\delta(q_2, q_3), a)$
$= \varepsilon\text{-closure}(\delta(q_2, a) \cup \delta(q_3, a))$
$= \phi$

$\hat{\delta}(q_2, b) = \varepsilon\text{-closure}(\delta(q_2, q_3), b)$
$= \varepsilon\text{-closure}(\delta(q_2, b) \cup \delta(q_3, b))$
$= \varepsilon\text{-closure}(\phi \cup \phi)$
$= \phi$

$\hat{\delta}(q_2, c) = \varepsilon\text{-closure}\,(\delta(q_2, q_3), c)$

$= \varepsilon\text{-closure}\,(\delta(q_2, c) \cup \delta(q_3, c))$

$= \varepsilon\text{-closure}\,(q_2)$

$= \{q_2, q_3\}$

$\hat{\delta}(q_2, d) = \varepsilon\text{-closure}\,(\delta(q_2, q_3), d)$

$= \varepsilon\text{-closure}\,(\delta(q_2, d) \cup \delta(q_3, d))$

$= \varepsilon\text{-closure}\,(\emptyset \cup q_3)$

$= \{q_3\}$

$\hat{\delta}(q_3, a) = \varepsilon\text{-closure}\,(\delta(q_3), a)$

$= \varepsilon\text{-closure}\,(\delta(q_3, a))$

$= \emptyset$

$\hat{\delta}(q_3, b) = \varepsilon\text{-closure}\,(\delta(q_3, b))$

$= \emptyset$

$\hat{\delta}(q_3, c) = \varepsilon\text{-closure}\,(\delta(q_3, c))$

$= \emptyset$

$\hat{\delta}(q_3, d) = \varepsilon\text{-closure}\,(\delta(q_3, d))$

$= \varepsilon\text{-closure}\,(q_3)$

$= \{q_3\}$

Introduction to Regular Expressions

Regular expressions is the language part of Type-3 grammar which is called "Regular Grammar".

* The Regular Expression is accepted by machine called "finite Automata"

Basics of Regular Expressions.

* Regular Expression can be defined as a language (or) stream accepted by a finite automata.

Some formal definitions of Regular Expressions:

1. Any terminals i.e., the symbols that belongs to Σ are regular expressions

Ex: i) $\Sigma = \{0, 1\}$
R.E = 0
R.E = 1

ii) $\Sigma = \{a, b, c\}$
a is R.E
b is R.E
c is R.E

* The null string (ϵ (or) λ) and null set (ϕ) are also regular expressions

2. If P & Q are two regular expressions then the union of the two regular expressions is denoted by

"P+Q", is also a regular expression.

3. If P & Q are two R.E then their concatenation denoted by "PQ" is also a R.E

4. If P is a R.E then the iteration (repetition) closure denoted by "P^*" is also a R.E.

$P^* = \{\varepsilon, P, PP, PPP, \ldots\}$

5. The R.Es got by the repeated application of the rules from 1 to 4 over Σ also R.E

KLEENE CLOSURE

It is defined as set of all strings including null obtained from the alphabet of the set 'Σ', it is denoted by Σ^*

Ex:-

1) $\Sigma = \{0\}$ then $\Sigma^* = \{\varepsilon, 0, 00, 000 \ldots\}$

2) $\Sigma = \{0,1\}$ then $\Sigma^* = \{\varepsilon, 0, 1, 10, 01, 11, 00, 000 \ldots\}$

Q. Describe the following R.E in english language.

(i) 10^*1

The strings that we get for R.E 10^*1 is

$L = \{11, 101, 1001, 10001, 100001, \ldots\}$

English: "The set of strings that are generated from the R.Es are beginning with '1' and ending with '1' in between 1 & 1 there are any no. of 0's.

(or)

"Set of all strings beginning & ending with '1' with any no. of 0's in between the two 1's."

(2) $0^* 1^*$

$0^* = \{\varepsilon, 0, 00, \ldots\}$

$1^* = \{\varepsilon, 1, 11, \ldots\}$

$L = \{\varepsilon, 0, 1, 01, 0011, 000111 \ldots\}$

English: "Set of all strings of 0's and 1's containing any no. of 0's followed by any no. of 1's."

(3) $(ab)^*$

$L = \{\varepsilon, ab, abab, ababab, abababab \ldots\}$.

In english it is described as

"Set of all strings of a's & b's with equal no. of a's and b's containing 'ab' as repetition".

(4) $(a^* + b^*)c^*$

$a^* = \{\varepsilon, a, aa, aaa \ldots\}$

$b^* = \{\varepsilon, b, bb, bbb \ldots\}$

$(a+b) = a$ (or) b.

$a^* + b^* = aaa$ (or) bbb.

English: "Any combination of a's (or) any combination of b's followed by any combination of c's.

(5) $(00)^* (11)^* 1$

$L = \{$ ~~000~~ $00111, 000011111, 00000011111111 \ldots \}$

"Set of all strings of 0's and 1's with even no. of 0's followed by odd no. of 1's.

Q1 Write a R.E of any combination of a's & b's, beginning with a & ending with b.

Sol

$a(a+b)^* b$

$(a+b)^* = \{\varepsilon, a, b, ab, ba, abb \ldots\}$

$\downarrow$

Any combination of a's & b's

Q2. Write a R.E of any combination of a's and b's containing abb

Sol

$(a+b)^* \, abb \, (a+b)^*$

Q3. Write a R.E of a's and b's having exactly one a

Sol

$b^* a b^*$.

Identity Rules of Regular Expression.

① $\emptyset + R = R + \emptyset = R$

L.H.S = { } + { elements of R }

= R

② $\emptyset R = R\emptyset = \emptyset$

L.H.S = { } $\wedge$ { elements of R }

= $\emptyset$

③ $\varepsilon R = R\varepsilon = R$

④ $\varepsilon^* = \varepsilon$ (or) $\wedge^* = \wedge$

⑤ $R + R = R$ ($R(\varepsilon + \varepsilon) = R$)

⑥ $R^* \cdot R^* = R^*$

⑦ $R^* R = R R^*$

L.H.S = $\{\varepsilon, R, RR, RRR \ldots\} R$

$= \{\varepsilon R, RR, RRR, RRRR \ldots\}$

$= R \{\varepsilon, R, RR \ldots\}$

$= RR^*$

= R.H.S

⑧ $(R^*)^* = R^*$

⑨ $\epsilon + RR^* = \epsilon + R^*R = R^*$

L.H.S = $\{\} + R\{\epsilon, R, RR, RRR \ldots\}$

$= \epsilon + \{R, RR, RRR, RRRR, \ldots\}$

$= \epsilon \cup \{\ldots\}$

$= \{\epsilon R, RR \ldots\}$

$= R^*$

⑩ $(PQ)^*P = P(QP)^*$

L.H.S = $(PQ)^*P = \{\epsilon, PQ, PQPQ, PQPQPQ \ldots\}P$

$= \{\epsilon P, PQP, PQPQP, PQPQPQP \ldots\}$

$= P\{\epsilon, QP, QPQP, QPQPQP \ldots\}$

$= P(QP)^*$

⑪ Demorgan's law

$(P+Q)^* = (P^*Q^*) = (P^*+Q^*)^*$

⑫ $(P+Q)R = PR + QR.$

Q) from the identities of R.E prove that

$(1+100^*) + (1+100^*)(0+10^*)(0+10^*)^* = 10^*(0+10^*)$

Sol

$(1+100^*) + (1+100^*)(0+10^*)(0+10^*)^*$

$= (1+100^*)(\epsilon + (0+10^*)(0+10^*)^*)$ [∵ take (1+100*) common]

$= (1+100^*)(0+10^*)$ [∵ $\epsilon + RR^* = R^*$]

$= 1(\epsilon + 00^*)(0+10^*)$ (Take 1 common)

$= 10^*(0+10^*)^*$ $[\because \varepsilon + RR^* = R^*]$

$=$ RHS.

Q2) $10 + (1010)^* [\varepsilon + (1010)^*] = 10 + (1010)^*$

sol

L.H.S

$= 10 + (1010)^* [\varepsilon + (1010)^*]$

$= 10 + \varepsilon(1010)^* + (1010)^*(1010)^*$ $[\because R^*R^* = R^*]$

$= 10 + (1010)^* + (1010)^*$ $[\because R+R=R]$

$= 10 + (1010)^*$

$=$ RHS.

Subject: ACD

Class Notes

Faculty: Bolleddu Devananda Rao

Topic: Conversion of Finite Automata to Regular expression

Unit No: I
Lecture No:
Link to Session Planner (SP):
Book Reference:
Date Conducted:
Page No: 23

Arden's theorem

If P, Q, R are R.E & if $R = P + RQ$

then R can be simplified as $R = PQ^*$

proof

$$R = P + PQ + RQ^*.$$

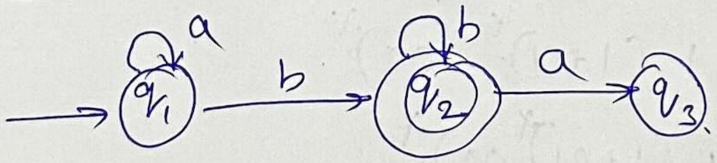
$$R = P + PQ + PQ^2 + PQ^3.$$

$$R = P + PQ + PQ^2 + PQ^3 + \cdots$$

$$R = P(\varepsilon + Q + Q^2 + Q^3 + \cdots$$

$$\boxed{R = PQ^*}$$

(1) Convert DFA into R.E.



$$q_2 = q_1 b + q_2 b.$$

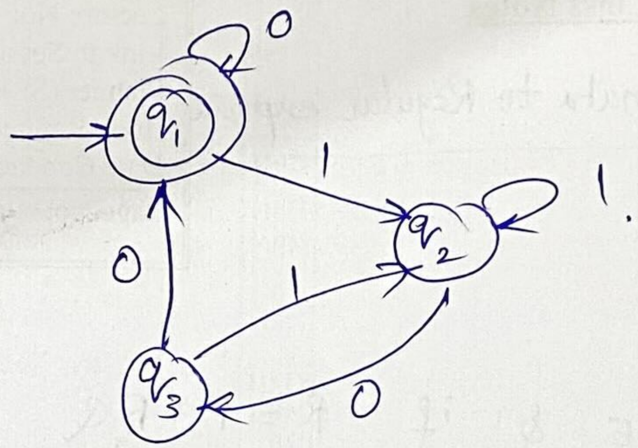
$$q_1 = q_1 a + \varepsilon.$$

$$\boxed{q_1 = a^*.}$$

$$q_2 = a^* b + q_2 b.$$

$$\boxed{q_2 = a^* b b^*}$$

(2)



(1) $q_1 = q_1 0 + \varepsilon + q_3 0$

(2) $q_2 = q_1 1 + q_2 1 + q_3 1$

(3) $q_3 = q_2 0$

$q_2 = q_1 1 + q_2 1 + q_2 01$

$q_2 = q_1 1 (1 + 01)^*$

$q_1 = q_1 0 + (q_2 0) 0 + \varepsilon$

$= q_1 (0 + (1 (1 + 01)^* 0) 0 + \varepsilon$

$q_1 = \varepsilon (0 + 1 (1 + 01)^* 00)^*$

$q_1 = (0 + 1 (1 + 01)^* 00)^*$